

1. JSP MVC Architecture

1.1 Overview of MVC Architecture

- **Definition:** The Model-View-Controller (MVC) architecture is a design pattern used in web applications to separate concerns, making applications easier to manage, scale, and test.
- **Components:**
 - **Model:** Represents the application's data and business logic.
 - **View:** Responsible for displaying data to the user.
 - **Controller:** Acts as an intermediary between the Model and the View.

1.2 Components of MVC Architecture

A. Model

- **Definition:** The Model handles data retrieval, processing, and business logic.
- **Example:**
 - **User Class:**

```
java
VerifyOpen In EditorRunCopy code
1public class User {
2    private int id;
3    private String name;
4    private String email;
5    // Getters and Setters
6}
```

- **User DAO Class:**

```
java
VerifyOpen In EditorRunCopy code
1public class UserDao {
2    public User getUser ById(int id) {
3        // Database logic to retrieve user by ID
4    }
5}
```

B. View

- **Definition:** The View displays data to the user, typically using JSP files.
- **Example:**
 - **user.jsp:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<html>
3<head>
4    <title>User Information</title>
5</head>
6<body>
7    <h1>User Information</h1>
8    <p>Name: ${user.name}</p>
9    <p>Email: ${user.email}</p>
10</body>
11</html>
```

C. Controller

- **Definition:** The Controller processes user input, interacts with the Model, and determines which View to display.
- **Example:**
 - **User Controller.java:**

```
java
VerifyOpen In EditorRunCopy code
1@WebServlet("/user")
2public class UserController extends HttpServlet {
3    protected void doGet(HttpServletRequest request,
4        HttpServletResponse response) throws ServletException, IOException {
5        int userId = Integer.parseInt(request.getParameter("id"));
6        UserDAO userDAO = new UserDAO();
7        User user = userDAO.getUserById(userId);
8        request.setAttribute("user", user);
9        RequestDispatcher dispatcher =
10        request.getRequestDispatcher("user.jsp");
11        dispatcher.forward(request, response);
12    }
13}
```

1.3 Flow of MVC Architecture

1. **User Interaction:** User interacts with the application (e.g., clicking a link).
2. **Request Handling:** The request is sent to the Controller.
3. **Model Interaction:** The Controller interacts with the Model to retrieve data.
4. **Data Preparation:** The Controller prepares the data and sets it as an attribute in the request.
5. **View Rendering:** The Controller forwards the request to the View, which generates the HTML response.
6. **Response to User:** The generated HTML is sent back to the user's browser.

1.4 Complete Example of JSP MVC Architecture

- **Directory Structure:**

```
VerifyOpen In EditorRunCopy code
1<webapps>
2├── mvcApp
3│   ├── WEB-INF
4│   │   ├── classes
5│   │   ├── lib
6│   │   └── web.xml
7│   ├── user.jsp
8│   └── UserController.java
```

1.5 web.xml Configuration

- **Example:**

```
xml
VerifyOpen In EditorRunCopy code
1<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
2    <servlet>
3        <servlet-name>UserController</servlet-name>
4        <servlet-class>com.example.UserController</servlet-class>
5    </servlet>
```

```

6    <servlet-mapping>
7        <servlet-name>UserController</servlet-name>
8        <url-pattern>/user</url-pattern>
9    </servlet-mapping>
10</web-app>

```

1.6 Conclusion

- The JSP MVC architecture effectively separates concerns, making applications easier to manage and maintain. It allows developers to create scalable and testable applications.
-

2. JSP Life Cycle

2.1 Overview of JSP Life Cycle

- **Definition:** The JSP life cycle consists of several phases from the creation to the destruction of a JSP page.

2.2 Phases of the JSP Life Cycle

1. **Translation Phase:** The JSP container translates the JSP page into a servlet.
 - **Example:** For a JSP file named `example.jsp`, it generates `example_jsp.java`.
2. **Compilation Phase:** The generated servlet code is compiled into a Java class file.
 - **Example:** `example_jsp.java` is compiled into `example_jsp.class`.
3. **Class Loading Phase:** The compiled servlet class is loaded into memory.
4. **Instantiation Phase:** An instance of the servlet class is created.
5. **Initialization Phase:** The `jspInit()` method is called to initialize the servlet instance.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1public void jspInit() {
2    System.out.println("JSP Page Initialized");
3}

```

6. **Request Processing Phase:** The `jspService()` method is called to handle requests.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1public void _jspService(HttpServletRequest request,
2    HttpServletResponse response) throws ServletException, IOException {
3    PrintWriter out = response.getWriter();
4    out.println("<h1>Hello, JSP!</h1>");
5}

```

7. **Destruction Phase:** The `jspDestroy()` method is called when the JSP page is being removed from service.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1public void jspDestroy() {
2    System.out.println("JSP Page Destroyed");
3}
```

2.3 Conclusion

- Understanding the JSP life cycle is essential for developing dynamic web applications, as it helps manage resources effectively.
-

3. JSP Directives

3.1 Overview of JSP Directives

- **Definition:** JSP directives provide global information about an entire JSP page and control the behavior of the JSP engine.

3.2 Types of JSP Directives

A. Page Directive

- **Purpose:** Defines attributes that apply to the entire JSP page.
- **Common Attributes:**
 - **import:** Specifies Java classes to be imported.
 - **contentType:** Sets the content type of the response.
 - **session:** Indicates whether the page can use session variables.
 - **isErrorPage:** Indicates whether the page can handle errors.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page import="java.util.Date" %>
2<%@ page contentType="text/html; charset=UTF-8" %>
3<%@ page session="true" %>
4<html>
5<body>
6    <h1>Current Date and Time</h1>
7    <p><%= new Date() %></p>
8</body>
9</html>
```

B. Include Directive

- **Purpose:** Includes the content of another file at translation time.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ include file="header.jsp" %>
2<html>
3<body>
```

```

4    <h1>Welcome to My Website</h1>
5    <p>This is the main content of the page.</p>
6</body>
7</html>

```

C. Taglib Directive

- **Purpose:** Declares a tag library for using custom tags in the JSP page.
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:if test="${not empty user}">
5        <h1>Welcome, ${user}!</h1>
6    </c:if>
7</body>
8</html>

```

3.3 Conclusion

- JSP directives are powerful tools for controlling the behavior of JSP pages, allowing developers to set page-wide properties and include other resources effectively.

4. JSP Implicit and Explicit Objects

4.1 Implicit Objects in JSP

- **Definition:** Implicit objects are pre-defined objects automatically available to all JSP pages.

4.2 List of JSP Implicit Objects

Object	Type	Description
out	JspWriter	Used to send output to the client.
request	HttpServletRequest	Represents the client's request.
response	HttpServletResponse	Represents the response sent to the client.
session	HttpSession	Represents the session for the user.
application	ServletContext	Represents the web application context.
config	ServletConfig	Provides configuration information for the JSP page.
pageContext	PageContext	Provides access to various scopes.
page	Object	Refers to the current JSP page instance.
exception	Throwable	Represents any exception that occurs in the JSP page.

4.3 Examples of Implicit Objects

Example 1: Using out Implicit Object

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>

```

```

2<html>
3<body>
4    <% out.print("Today's date is: " + new java.util.Date()); %>
5</body>
6</html>

```

Example 2: Using request Implicit Object

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<body>
4    <form action="welcome.jsp" method="post">
5        Name: <input type="text" name="username">
6        <input type="submit" value="Submit">
7    </form>
8</body>
9</html>

```

In welcome.jsp:

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<body>
4    <% String username = request.getParameter("username"); %>
5    <p>Welcome, <%= username %>!</p>
6</body>
7</html>

```

Example 3: Using session Implicit Object

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<body>
4    <% session.setAttribute("user", "John Doe"); %>
5    <p>User session created for: <%= session.getAttribute("user") %></p>
6</body>
7</html>

```

4.4 Explicit Objects in JSP

- **Definition:** Explicit objects are user-defined objects created and managed within the JSP page.

4.5 Example of Using a JavaBean in JSP

JavaBean Class:

```

java
VerifyOpen In EditorRunCopy code
1package com.example;
2

```

```

3public class User {
4    private String name;
5    private int age;
6
7    // Getters and Setters
8    public String getName() { return name; }
9    public void setName(String name) { this.name = name; }
10   public int getAge() { return age; }
11   public void setAge(int age) { this.age = age; }
12}

```

Using JavaBean in JSP:

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<%@ page import="com.example.User" %>
3<html>
4<body>
5    <jsp:useBean id="user" class="com.example.User" scope="session" />
6    <jsp:setProperty name="user" property="name" value="John Doe" />
7    <jsp:setProperty name="user" property="age" value="30" />
8
9    <h1>User Information</h1>
10   <p>Name: <jsp:getProperty name="user" property="name" /></p>
11   <p>Age: <jsp:getProperty name="user" property="age" /></p>
12</body>
13</html>

```

4.6 Conclusion

- Understanding both implicit and explicit objects in JSP is essential for developing dynamic web applications, allowing for effective data management.
-

5. JSP Scriptlets

5.1 Overview of JSP Scriptlets

- **Definition:** JSP scriptlets are blocks of Java code embedded within a JSP page, allowing for dynamic content generation.

5.2 Key Features of Scriptlets

- **Dynamic Content Generation:** Enables the generation of content based on server-side logic.
- **Java Code Execution:** Any valid Java code can be written inside scriptlets.
- **Output to Client:** The output generated by scriptlets can be sent to the client using the `out` implicit object.

5.3 Syntax of Scriptlets

```

jsp
VerifyOpen In EditorRunCopy code
1<%

```

```
2    // Java code goes here
3%>
```

5.4 Examples of JSP Scriptlets

Example 1: Basic Scriptlet

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<head>
4    <title>Current Date and Time</title>
5</head>
6<body>
7    <h1>Current Date and Time</h1>
8    <%
9        java.util.Date date = new java.util.Date();
10       out.println("The current date and time is: " + date);
11    %>
12</body>
13</html>
```

Example 2: Using Variables in Scriptlets

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<head>
4    <title>Simple Calculation</title>
5</head>
6<body>
7    <h1>Simple Calculation</h1>
8    <%
9        int a = 10;
10       int b = 20;
11       int sum = a + b;
12       out.println("The sum of " + a + " and " + b + " is: " + sum);
13    %>
14</body>
15</html>
```

Example 3: Conditional Logic in Scriptlets

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<head>
4    <title>Conditional Logic</title>
5</head>
6<body>
7    <h1>Check Even or Odd</h1>
8    <%
9        int number = 15; // Change this value to test
10       if (number % 2 == 0) {
11           out.println(number + " is an even number.");
12       } else {
```

```

13         out.println(number + " is an odd number.");
14     }
15     %>
16</body>
17</html>

```

Example 4: Looping with Scriptlets

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<head>
4    <title>Looping Example</title>
5</head>
6<body>
7    <h1>Numbers from 1 to 10</h1>
8    <ul>
9        <%
10            for (int i = 1; i <= 10; i++) {
11                out.println("<li>" + i + "</li>");
12            }
13        %>
14    </ul>
15</body>
16</html>

```

5.5 Best Practices for Using Scriptlets

- **Minimize Scriptlet Usage:** Use scriptlets sparingly; prefer JSTL for logic and control flow.
- **Separate Logic from Presentation:** Keep business logic in servlets or Java classes.
- **Use EL for Output:** Prefer Expression Language for outputting data instead of scriptlets.

5.6 Conclusion

- JSP scriptlets provide a way to embed Java code directly into JSP pages, allowing for dynamic content generation. However, modern approaches like JSTL and EL are preferred for cleaner code.
-

6. JSP Expressions

6.1 Overview of JSP Expressions

- **Definition:** JSP expressions are a shorthand way to output a single Java expression to the client.

6.2 Syntax of JSP Expressions

```

jsp
VerifyOpen In EditorRunCopy code
1<%= expression %>

```

- **Note:** The expression does not end with a semicolon.

6.3 Examples of JSP Expressions

Example 1: Basic Expression

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<html>
3<head>
4    <title>JSP Expression Example</title>
5</head>
6<body>
7    <h1>Welcome to JSP Expressions!</h1>
8    <p>Current Date and Time: <%= new java.util.Date() %></p>
9</body>
10</html>
```

Example 2: Using Variables in Expressions

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<% String userName = "Alice"; %>
3<html>
4<head>
5    <title>User Greeting</title>
6</head>
7<body>
8    <h1>Hello, <%= userName %>!</h1>
9</body>
10</html>
```

Example 3: Performing Calculations

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<html>
3<head>
4    <title>Calculation Example</title>
5</head>
6<body>
7    <h1>Simple Calculation</h1>
8    <p>The sum of 5 and 10 is: <%= 5 + 10 %></p>
9</body>
10</html>
```

Example 4: Accessing Request Parameters

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<html>
3<head>
4    <title>Request Parameter Example</title>
5</head>
6<body>
7    <h1>Welcome!</h1>
8    <p>Your name is: <%= request.getParameter("name") %></p>
9</body>
10</html>
```

- **Usage:** Access the JSP page with a URL like: `http://localhost:8080/yourapp/example.jsp?name=John`.

Example 5: Using Expressions with Conditional Logic

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<% boolean isLoggedIn = true; %>
3<html>
4<head>
5    <title>Conditional Expression Example</title>
6</head>
7<body>
8    <h1><%= isLoggedIn ? "Welcome back!" : "Please log in." %></h1>
9</body>
10</html>
```

6.4 Conclusion

- JSP expressions provide a simple and effective way to embed Java code within HTML, allowing for dynamic content generation.
-

7. JSP Expression Language (EL)

7.1 Overview of JSP Expression Language

- **Definition:** JSP Expression Language (EL) simplifies data access in JSP pages without embedding Java code directly.

7.2 Key Features of JSP Expression Language

- **Simplified Syntax:** EL uses a simple syntax to access data.
- **Automatic Type Conversion:** EL automatically converts data types as needed.
- **Support for Collections:** EL can access properties of JavaBeans, elements of arrays, and entries in maps.
- **Logical and Arithmetic Operations:** EL supports basic arithmetic and logical operations.

7.3 Basic Syntax of EL

```
jsp
VerifyOpen In EditorRunCopy code
1${expression}
```

7.4 Examples of JSP Expression Language

Example 1: Accessing JavaBeans Properties

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<%@ page import="com.example.User" %>
3<jsp:useBean id="user" class="com.example.User" scope="session" />
```

```

4<jsp:setProperty name="user" property="name" value="John Doe" />
5<jsp:setProperty name="user" property="email" value="john@example.com" />
6<html>
7<body>
8    <h1>User Information</h1>
9    <p>Name: ${user.name}</p>
10   <p>Email: ${user.email}</p>
11</body>
12</html>

```

Example 2: Accessing Request Parameters

```

jsp
VerifyOpen In EditorRunCopy code
1<html>
2<body>
3    <form action="process.jsp" method="post">
4        Name: <input type="text" name="username">
5        <input type="submit" value="Submit">
6    </form>
7</body>
8</html>

```

In process.jsp:

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<body>
4    <h1>Welcome, ${param.username}!</h1>
5</body>
6</html>

```

Example 3: Using Collections

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
3<%
4    List<User> userList = new ArrayList<>();
5    userList.add(new User("Alice", "alice@example.com"));
6    userList.add(new User("Bob", "bob@example.com"));
7    request.setAttribute("userList", userList);
8%>
9<html>
10<body>
11    <h1>User List</h1>
12    <ul>
13        <c:forEach var="user" items="${userList}">
14            <li>${user.name} - ${user.email}</li>
15        </c:forEach>
16    </ul>
17</body>
18</html>

```

Example 4: Performing Arithmetic Operations

```

jsp

```

```
VerifyOpen In EditorRunCopy code
1<%
2    int a = 10;
3    int b = 20;
4    request.setAttribute("sum", a + b);
5%>
6<html>
7<body>
8    <h1>Sum: ${sum}</h1>
9</body>
10</html>
```

Example 5: Conditional Expressions

```
jsp
VerifyOpen In EditorRunCopy code
1<%
2    boolean isLoggedIn = true; // Simulating user login status
3    request.setAttribute("isLoggedIn", isLoggedIn);
4%>
5<html>
6<body>
7    <h1>${isLoggedIn ? 'Welcome back!' : 'Please log in.'}</h1>
8</body>
9</html>
```

7.5 Conclusion

- JSP Expression Language (EL) simplifies data access and manipulation in JSP pages, allowing for cleaner and more maintainable code.
-

8. JSP Scope

8.1 Overview of JSP Scope

- **Definition:** Scope refers to the visibility and lifetime of objects stored in various contexts.

8.2 Types of JSP Scopes

A. Page Scope

- **Definition:** Objects stored in page scope are accessible only within the current JSP page.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%
2    String message = "Hello, Page Scope!";
3    pageContext.setAttribute("pageMessage", message);
4%>
5<p>Message: <%= pageContext.getAttribute("pageMessage") %></p>
```

B. Request Scope

- **Definition:** Objects stored in request scope are available during the request-response cycle.
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%
2    String requestMessage = "Hello, Request Scope!";
3    request.setAttribute("requestMessage", requestMessage);
4%>
5<p>Message: <%= request.getAttribute("requestMessage") %></p>

```

C. Session Scope

- **Definition:** Objects stored in session scope remain available until the session is invalidated or expires.
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%
2    String sessionMessage = "Hello, Session Scope!";
3    session.setAttribute("sessionMessage", sessionMessage);
4%>
5<p>Message: <%= session.getAttribute("sessionMessage") %></p>

```

D. Application Scope

- **Definition:** Objects stored in application scope remain available until the application is shut down or the object is explicitly removed.
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%
2    String appMessage = "Hello, Application Scope!";
3    application.setAttribute("appMessage", appMessage);
4%>
5<p>Message: <%= application.getAttribute("appMessage") %></p>

```

8.3 Summary of JSP Scopes

Scope	Lifetime	Visibility
Page Scope	Until the page is processed	Only within the current JSP page
Request Scope	Until the response is sent	Only during the request-response cycle
Session Scope	Until the session is invalidated	Across multiple requests from the same user
Application Scope	Until the application is stopped	Across the entire web application

8.4 Conclusion

- Understanding the different scopes in JSP is crucial for managing data effectively in web applications.
-

9. JSP Error Handling

9.1 Overview of Error Handling in JSP

- **Definition:** Error handling in JSP allows developers to manage exceptions and provide user-friendly error messages.

9.2 Using `errorPage` and `isErrorPage` Attributes

- **`errorPage` Attribute:** Specifies the URL of the JSP page to display when an error occurs.
- **`isErrorPage` Attribute:** Indicates that the page is designed to handle errors.

9.3 Example of JSP Error Page Handling

Step 1: Create the Main JSP Page

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page errorPage="errorPage.jsp" %>
2<html>
3<head>
4    <title>Welcome Page</title>
5</head>
6<body>
7    <h1>Welcome to the Application</h1>
8    <p>Result of division: <%= 10 / 0 %></p> <!-- This will cause an
ArithmeticException -->
9</body>
10</html>
```

Step 2: Create the Error Page

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page isErrorPage="true" %>
2<html>
3<head>
4    <title>Error Page</title>
5</head>
6<body>
7    <h1>An Error Occurred</h1>
8    <p>Sorry, an error has occurred while processing your request.</p>
9    <p>Exception: <%= exception.getMessage() %></p> <!-- Display the exception
message -->
10</body>
11</html>
```

9.4 How It Works

1. When a user accesses `welcome.jsp`, the division by zero occurs, causing an `ArithmeticException`.
2. The JSP engine detects the error and redirects the request to `errorPage.jsp`.
3. In `errorPage.jsp`, the `isErrorPage` attribute indicates that this page is designed to handle errors.

9.5 Conclusion

- Using the `errorPage` and `isErrorPage` attributes allows developers to create user-friendly error handling mechanisms in JSP applications.
-

10. Error Handling Using `<error - page>` Element in `web.xml`

10.1 Overview of `<error - page>` Element

- **Definition:** The `<error - page>` element in `web.xml` defines custom error pages for specific HTTP error codes or exceptions.

10.2 Syntax of `<error - page>` Element

```
xml
VerifyOpen In EditorRunCopy code
1<error-page>
2    <error-code>error_code</error-code>
3    <location>/path/to/errorPage.jsp</location>
4</error-page>
```

10.3 Example of Configuring Error Pages in `web.xml`

```
xml
VerifyOpen In EditorRunCopy code
1<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
2    <error-page>
3        <error-code>404</error-code>
4        <location>/error404.jsp</location>
5    </error-page>
6    <error-page>
7        <error-code>500</error-code>
8        <location>/error500.jsp</location>
9    </error-page>
10    <error-page>
11        <exception-type>java.lang.Exception</exception-type>
12        <location>/error.jsp</location>
13    </error-page>
14</web-app>
```

10.4 Creating Custom Error Pages

`error404.jsp`

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<head>
4    <title>404 Not Found</title>
5</head>
6<body>
7    <h1>404 - Page Not Found</h1>
```

```

8      <p>Sorry, the page you are looking for does not exist.</p>
9      <a href="index.jsp">Go to Home Page</a>
10</body>
11</html>

```

error500.jsp

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
2<html>
3<head>
4      <title>500 Internal Server Error</title>
5</head>
6<body>
7      <h1>500 - Internal Server Error</h1>
8      <p>Sorry, something went wrong on our end. Please try again later.</p>
9      <a href="index.jsp">Go to Home Page</a>
10</body>
11</html>

```

error.jsp

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" isErrorPage="true"%>
2<html>
3<head>
4      <title>Error Occurred</title>
5</head>
6<body>
7      <h1>An Error Occurred</h1>
8      <p>Error Message: <%= exception.getMessage() %></p>
9      <p>Please contact support.</p>
10      <a href="index.jsp">Go to Home Page</a>
11</body>
12</html>

```

10.5 Testing Error Handling

- Deploy your JSP application with the configured `web.xml` and error pages.
- Access a non-existent page to trigger a 404 error: `http://localhost:8080/yourapp/nonexistentpage.jsp`.
- Accessing `http://localhost:8080/yourapp/testError` will redirect you to `error.jsp`, displaying the error message.

10.6 Conclusion

- Using the `<error-page>` element in `web.xml` allows you to define custom error handling in your JSP applications, enhancing user experience.
-

11. JSTL Core Functionalities

11.1 Overview of JSTL

- **Definition:** JavaServer Pages Standard Tag Library (JSTL) is a collection of custom tags that simplify the development of JSP pages.

11.2 Key Core JSTL Tags

A. c:out Tag

- **Purpose:** Outputs data to the client, escaping HTML characters to prevent XSS attacks.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:set var="username" value="John Doe" />
5    <h1>Welcome, <c:out value="${username}" /></h1>
6</body>
7</html>
```

B. c:if Tag

- **Purpose:** Conditionally includes content based on a specified condition.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:set var="isLoggedIn" value="true" />
5    <c:if test="${isLoggedIn}">
6        <h1>Welcome back!</h1>
7    </c:if>
8    <c:if test="${!isLoggedIn}">
9        <h1>Please log in.</h1>
10    </c:if>
11</body>
12</html>
```

C. c:choose, c:when, c:otherwise Tags

- **Purpose:** Implements a switch-case-like structure in JSP.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:set var="role" value="admin" />
5    <c:choose>
6        <c:when test="${role == 'admin'}">
7            <h1>Welcome, Admin!</h1>
8        </c:when>
```

```

9      <c:when test="${role == 'user'}">
10          <h1>Welcome, User!</h1>
11      </c:when>
12      <c:otherwise>
13          <h1>Welcome, Guest!</h1>
14      </c:otherwise>
15  </c:choose>
16</body>
17</html>

```

D. c:forEach Tag

- **Purpose:** Iterates over a collection (like a list or an array).
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core " prefix="c" %>

```

```

<% List fruits = Arrays.asList("Apple", "Banana", "Cherry"); request.setAttribute("fruits", fruits);
%>

```

Fruits List

- \${fruit}

...

E. c:forTokens Tag

- **Purpose:** Iterates over a string that is tokenized based on a specified delimiter.
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:set var="colors" value="Red,Green,Blue" />
5    <h1>Colors List</h1>
6    <ul>
7        <c:forTokens var="color" items="${colors}" delims=", ">
8            <li>${color}</li>
9        </c:forTokens>
10    </ul>
11</body>
12</html>

```

F. c:set Tag

- **Purpose:** Sets a variable in a specified scope (page, request, session, or application).
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:set var="username" value="Alice" scope="session" />

```

```
5    <h1>Hello, ${username}!</h1>
6</body>
7</html>
```

G. c:remove Tag

- **Purpose:** Removes a variable from a specified scope.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:set var="username" value="Alice" scope="session" />
5    <c:remove var="username" scope="session" />
6    <c:if test="${empty username}">
7        <h1>Username has been removed!</h1>
8    </c:if>
9</body>
10</html>
```

H. c:import Tag

- **Purpose:** Includes content from another resource (like a servlet or JSP) dynamically.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<html>
3<body>
4    <c:import url="header.jsp" />
5    <h1>Main Content</h1>
6    <c:import url="footer.jsp" />
7</body>
8</html>
```

I. c:redirect Tag

- **Purpose:** Redirects the client to a different URL.
- **Example:**

```
jsp
VerifyOpen In EditorRunCopy code
1<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2<c:redirect url="welcome.jsp" />
```

11.3 Conclusion

- JSTL provides a powerful set of tags that simplify JSP development by allowing for easier control of flow and data management, making JSP pages cleaner and more maintainable.

This comprehensive overview of JSP concepts, including MVC architecture, life cycle, directives, implicit and explicit objects, scriptlets, expressions, expression language, scopes, error handling,

and JSTL, should provide a solid foundation for understanding and working with JSP in web applications. If you have any further questions or need more examples, feel free to ask!